

多模态大模型与智能体驱动的脑病跨尺度分析

以脑细胞外间隙（ECS）研究为例

冉源潮

2026年7月

摘要

脑细胞外间隙（ECS）由细胞外液、细胞外基质及其曲折微通道构成，参与离子稳态、神经活性物质扩散与代谢物清除。其微尺度结构难以被单一临床检查直接测量，因此脑病研究需要整合影像、病理、组学、时序生理信号和临床文本等异构证据。本文区分多模态大模型的联合表征能力与智能体的受约束工具编排能力，讨论二者在ECS相关脑病研究和临床辅助中的价值、局限及可验证实现路径。核心主张是：其价值不在于跨越尺度直接给出确定性诊断，而在于构建受物理规律与临床证据约束的假设生成与验证空间。为降低死后组织、动物实验与活体MRI之间的域偏移风险，框架将不确定性量化纳入跨尺度映射；面对真实临床中的缺失模态，智能体以预期信息增益而非模态完整性为目标，提出可由医生审核的补充检查建议。通过显式保留尺度差异、不确定性、证据来源和人类复核权限，该框架可为ECS相关机制研究与临床辅助提供可追溯的计算路径。

关键词：脑细胞外间隙；多模态学习；大语言模型；智能体；不确定性量化；人在回路

目录

1 引言	3
2 概念与核心技术架构	3
2.1 大模型、多模态模型与智能体的边界	3
2.2 生物学优势与必要约束	3
3 阿尔茨海默病：ECS清除障碍的多模态闭环	3
3.1 从类淋巴交换到A β /Tau蓄积的可检验问题	3
3.2 AD多模态价值矩阵	4
4 ECS相关多模态数据及应用价值	4
5 可实现且可验证的技术路线	5
5.1 跨尺度对齐：从微观组织图到MRI体素	5
5.2 数据与建模	5
5.2.1 合成数据演示与消融结果	6
5.3 ECS跨尺度数字孪生智能体：工具链与验证	6

5.4	临床交互与人机协同	7
5.5	假设生成与Agent评估体系	8
6	挑战、伦理与展望	8
7	结论	9
A	可复现代码	9
A.1	运行配置	9
A.2	合成数据生成	9
A.3	PINN示踪剂反演演示	10
A.4	ECS样式分割基线	12
A.5	消融实验	14
A.6	结果绘制	16

1 引言

ECS并非细胞之间的被动空隙，而是神经元、胶质细胞和血管相关细胞之间的动态传质空间。其体积分数 α 、曲折度 λ 与有效扩散系数 D^* 常用于描述局部运输受限程度，经典近似为 $D^* = D/\lambda^2$ ^[1-2]。活脑超分辨成像提示，ECS在微尺度上具有显著空间异质性^[3]；不同物种、状态、测量方法及空间尺度下的参数不宜直接互换。

脑水肿和缺血可伴随细胞肿胀与细胞外体积改变；创伤、癫痫、神经退行性病变、肿瘤微环境及炎症也可能重塑细胞外基质或间质运输。上述关联是研究问题而非自动成立的因果链，仍需纵向影像、组织学与干预实验共同验证。脑脊液-间质液交换及类淋巴相关机制具有重要研究价值，但其路径和驱动因素仍存在方法学争议^[4-5]。

2 概念与核心技术架构

2.1 大模型、多模态模型与智能体的边界

大模型通过大规模预训练获得可迁移表征或生成能力；多模态模型进一步从影像、文本、结构化变量、波形或组学等输入学习联合表示；智能体则是在明确权限、工具与停止规则下，调用模型、数据库或分析软件完成多步骤任务的执行系统。三者不应混同：联合表示不能保证物理规律成立，工具调用也不能替代临床证据判断。

一个面向ECS的系统可由五层组成：（1）模态专属编码器，将MRI、病理切片、EEG/MEG、表格和文本转为表示；（2）以受试者、脑区、采集时点和协议为键的跨模态对齐与融合；（3）检索增强生成（RAG），使生成性说明链接至版本化文献与本地数据字典^[6]；（4）智能体编排质控、分割、统计、物理反演和报告工具，并保留调用日志；（5）人类复核、访问控制与不确定性呈现。ReAct等框架说明了推理-行动-观察的工具协作范式^[7]，但医疗任务仍须限制工具权限与输出范围。

2.2 生物医学优势与必要约束

多模态方法可将影像的空间信息、病历文本的病程信息、实验记录的条件信息和组学的分子信息置于同一可查询框架；基础模型也可能降低小样本任务的标注需求^[8]。但缺失模态、中心扫描协议差异、选择偏倚、标签噪声和域迁移均可造成性能虚高。生成模型还可能产生无来源的幻觉，相关性模型不能据此宣称因果关系。临床部署前必须报告校准度、亚组表现、失败模式与外部验证结果，并满足隐私、伦理和全生命周期变更管理要求^[9-10]。

3 阿尔茨海默病：ECS清除障碍的多模态闭环

3.1 从类淋巴交换到 $A\beta$ /Tau蓄积的可检验问题

阿尔茨海默病（AD）为本文重点场景。脑脊液-间质液交换及血管周围通路可能参与可溶性代谢物清除；星形胶质细胞足突中AQP4的血管周围极化被提出为影响该过程的关键因素之一^[4,11]。人脑死后组织研究报告，AQP4定位改变与AD病理负荷相关^[11]；这提示血管周围微环境重塑-间质运输改变- $A\beta$ /Tau负荷是值得检验的跨尺度假设，而非已由单一指标证实的因果序列。睡

眠、衰老、血管危险因素和炎症均可能构成混杂或效应修饰因素，应在队列设计中预先记录和建模^[5,12]。

具体而言，问题可表述为：在年龄、睡眠、血管危险因素和疾病阶段可比的个体中，宏观扩散代理、血管周围间隙表型、 $A\beta$ /Tau体液标志物与AQP4相关微观证据能否形成一致的可计算联合表型？若能，该表型能否预测纵向认知或病理标志物变化？该问题将类淋巴清除受损拆解为可测量、可反驳的子问题，并为动物模型干预提供分层假设和明确终点。

3.2 AD多模态价值矩阵

表 1: 以AD的ECS/类淋巴清除障碍为例的跨尺度证据矩阵

证据层级	代表数据与可提取特征	与清除假设的关系	验证或解释边界
宏观影像	T1/FLAIR的PVS表型；DWI/DTI中DTI-ALPS扩散代理；动态对比增强或灌注数据（如有）	描述血管周围和水扩散的个体、纵向表型	DTI-ALPS不是直接的类淋巴流量或ECS几何测量；需控制扫描协议 ^[13]
分子与临床	CSF或血液 $A\beta$ 、p-tau、总Tau；认知量表、睡眠和血管危险因素	提供病理负荷、病程与混杂因素的时序锚点	标志物变化不等同于某一清除通路的特异性改变
微观组织与空间组学	AQP4极化、血管周围基底膜、 $A\beta$ 沉积；空间转录组中的星形胶质细胞/血管周围细胞状态	提供宏观影像表型的组织学候选解释	多来自死后组织或动物；不能逐例直接外推至活体MRI
智能体产物	带来源的影像-分子-微观证据联合表型与待检验机制图	发现一致或矛盾证据，生成分层与随访假设	不是诊断标签；结论须由独立队列和干预研究检验

例如，智能体可在DTI质控通过后提取DTI-ALPS等影像代理，链接同一受试者的 $A\beta$ /Tau时间序列、睡眠与血管危险因素；再从版本化知识库检索AQP4极化和血管周围基底膜的空间组学或病理证据。输出不应写成患者清除功能下降的确定性结论，而应是与清除受损假设一致的分子-影像联合表型，并列出现替代解释、数据缺口和下一步验证方案。模型的价值在于将分散证据转化为可追溯的研究问题。

4 ECS相关多模态数据及应用价值

MRI的DWI/ADC、DTI和DTI-ALPS等指标可提供水扩散或血管周围方向扩散的宏观代理，但不能表述为对ECS几何参数的直接测量^[13]。病理和活体示踪可补充微尺度信息，CSF/血液检测与组学可描述分子环境，病历与量表可锚定功能结局。合理融合应显式保留尺度、时间差和缺失状态，而不是把一种模态翻译为另一种模态。

在科研中，系统可筛选结构改变-运输表型-分子通路的关联模式，形成待验证假设。在临床中，系统可辅助汇总影像、病史和检验资料，提示低置信输入、缺失检查及与证据不一致之处；它不应取代放射科、神经科或病理医师的判断。

表 2: 多模态数据在ECS相关脑病研究中的任务、价值与验证要求

数据模态	可回答的问题（示例）	科研或临床辅助价值	风险与验证方式
MRI: DWI/ADC、 T1/T2、 FLAIR、 DTI、灌注 CT、PET	水扩散、结构损伤、血流 与白质微结构的宏观代理	病灶定位、分层和纵向表 型	不是ECS直接测量；需协 议一致性、重复扫描和外 部队列验证
EEG/MEG与 生理波形	神经网络活动及其时间关 系	癫痫等动态表型研究	辐射、时间不同步和非特 异性；需临床金标准核对 伪迹和空间定位限制；需 盲法事件标注
病理、动物实 验、ECS动态 示踪	细胞-基质边界及局部传质 假设	机制验证与参数先验	与人体宏观影像存在尺度 鸿沟；需独立重复与预注 册
病历、检验、 脑脊液、组学 与文献	病程、炎症/代谢背景和候 选通路	表型定义、假设生成和证 据追溯	混杂、批次效应和隐私风 险；需数据字典、去标识 与审计

5 可实现且可验证的技术路线

5.1 跨尺度对齐：从微观组织图到MRI体素

病理切片或空间转录组的分辨率与MRI体素相差数个数量级，不能直接拼接后交由交叉注意力处理。可先建立**分层中间表示**：在同一脑区的组织学图像中分割血管、星形胶质细胞、细胞外基质和沉积物，构建细胞-血管-基质异质图；节点包含AQP4极化、基底膜/沉积负荷和局部转录特征，边表示空间邻接或血管周围关系。再以脑区、层级和解剖坐标为锚点，将细胞类型比例、图谱模块分数和连通性聚合为虚拟组织图谱。图神经网络可将微观图的节点信息映射为脑区级特征，并与MRI体素级表征以层级对比学习对齐^[14]。

跨尺度映射的主要风险不是网络容量不足，而是域偏移。死后组织的自溶、固定和切片形变，AD相关脑萎缩、动物与人脑的血管及胶质结构差异，均可改变形态学特征的含义。对无配对的活体MRI与死后/动物空间组学，模型至多学习群体级先验，必须显式标注非个体配对，不得生成虚构的患者级空间转录组。建议以贝叶斯分层推断输出映射后验分布，并以Dempster-Shafer证据理论区分支持、反对和无知质量^[15]；同时报告中心、物种、死后间隔和萎缩程度分层的域外不确定性。医疗深度学习中的UQ还应同时检验校准度、分布外检测和风险-覆盖率曲线，避免仅以置信分数代替可靠性^[16]。若映射置信度低或证据冲突，应退回脑区级假设，或拒绝给出微观解释，而非以单一点估计掩盖系统性偏差。

5.2 数据与建模

首先预先定义病例/样本队列、纳入排除标准、目标任务、主要终点及数据使用权限。执行DICOM完整性检查、影像伪迹控制、病理染色批次校正、组学批次校正和缺失机制标记；训练、调参与测试集须按患者和中心严格隔离。模型可采用模态专属预训练编码器与交叉注意力融合，并以掩码训练提高缺失模态鲁棒性。输出应包括来源模态、置信区间或预测集合、适用域判断和不能回答

的问题。

对示踪浓度 C 的研究性反演，可将扩散-对流-清除模型作为待检验约束：

$$\frac{\partial C}{\partial t} = \nabla \cdot (D \nabla C) - \mathbf{v} \cdot \nabla C - kC + Q/\alpha. \quad (1)$$

其中 D 、 $\mathbf{v}$ 、 k 和 Q 分别表示有效扩散、速度场、清除项和源项。物理信息神经网络（PINN）可在观测与方程残差间联合拟合^[17]，从而使候选解释受质量守恒、边界条件和参数范围约束；但参数可辨识性、边界条件与模型错设必须通过模拟、独立测量和敏感性分析报告。其结果只能是模型条件下的参数区间，不能被解释为MRI对微观ECS参数的直接测量。

5.2.1 合成数据演示与消融结果

为使上述技术路线具有可复现的最小实现，附录代码生成一维示踪剂浓度序列和二维ECS样式切片，不读取人类、动物或公开影像。示踪剂生成器设定归一化的 $D = 0.018$ 、 $v = 0.12$ 和 $\alpha = 0.20$ ，并在两个合成区域引入不同扩散尺度；分割网络仅在120幅程序生成的 64×64 图像上学习区分细胞样椭圆与其间的ECS样区域。因此，以下结果只用于检验优化行为和代码链路，不能作为ECS测量精度、跨尺度转译能力或临床性能的证据。

三次随机种子的消融实验显示，数据项、PDE残差、初始条件和区域先验的组合在合成留出集上均得到约 1.00×10^{-3} 至 1.07×10^{-3} 的均方误差。加入PDE项后，均方根方程残差由0.01631降至0.00484；在初始条件和合成区域先验同时纳入时， D 的平均相对误差由0.37353降至0.34618。但 v 的相对误差仍为0.57260，说明在这一稀疏、低维的合成设定中，较小的预测误差或方程残差并不自动意味着所有反演参数可辨识。该现象支持正文的主张：PINN输出应以参数区间、敏感性分析和可证伪假设呈现，而非作为确定性微观参数。

表 3: 合成PINN消融实验的三次随机种子均值。所有数值仅描述程序生成数据上的方法学演示。

配置	留出集MSE	D 相对误差	v 相对误差	PDE残差
仅数据项	0.00106	0.37353	0.66667	0.01631
数据项加PDE	0.00107	0.53144	0.81685	0.00484
数据项加PDE和初始条件	0.00100	0.50845	0.57136	0.00671
数据项加PDE、初始条件和区域先验	0.00100	0.34618	0.57260	0.00679

5.3 ECS跨尺度数字孪生智能体：工具链与验证

可将系统实现为ECS跨尺度数字孪生智能体：它不是复制患者大脑的确定性模型，而是维护可更新的个体观测、物理参数区间、组织先验和机制假设。输入层接收T1/FLAIR、DWI/DTI（含DTI-ALPS代理）、体液标志物及可用的病理/动物空间组学；知识层挂载ECS方程库、 $A\beta$ 清除和AQP4相关的版本化文献库。智能体仅在以下ECS特异性白名单中调用工具：

1. DTI运动、涡流和协议质控工具；若质控失败，终止反演并请求重采或人工复核。
2. 基于随机游走或蒙特卡洛扩散模拟的曲折度、有效扩散敏感性分析工具，用于比较候选 λ 和 D^* 区间，而非从常规MRI直接宣称精确微观参数。

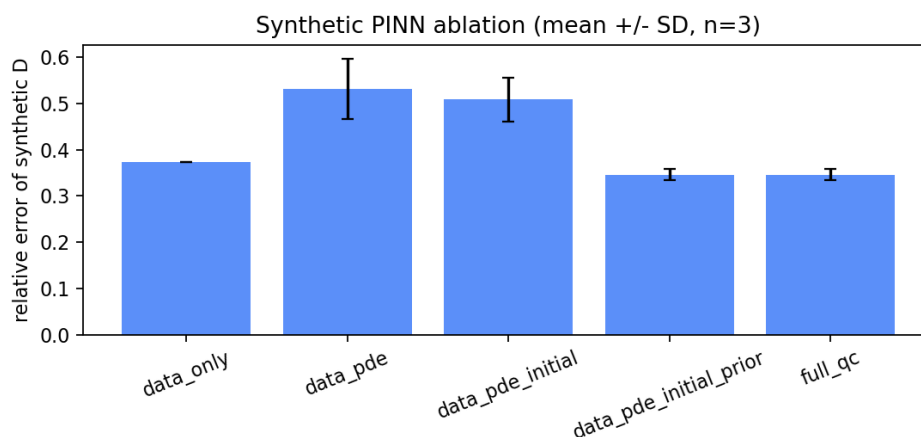


图 1: 合成PINN消融实验中 D 相对误差的均值和标准差, $n = 3$ 。该图反映合成训练域内的优化差异, 不代表人体ECS参数估计误差。

3. 空间组学工具（如Seurat/Squidpy工作流），用于分析AQP4相关星形胶质细胞状态、血管周围邻域与基质重塑模块^[18-19]。
4. PK/PD或对流-扩散求解器，用于研究性评估示踪剂或候选药物在假设ECS参数下的暴露分布；肿瘤药物递送场景仍须另行接受药代和安全性验证。
5. 因果图与反事实分析工具：基于预先指定的变量关系检查睡眠、年龄、血管危险因素等混杂路径，并提出最具信息量的随访或动物干预实验。
6. **主动信息增益计算与模态推荐工具**：当DTI-ALPS、CSF标志物或其他关键模态缺失时，计算候选检查在当前假设集合下预期减少的不确定性，并扣除检查风险、成本、可及性和时效性。只有净信息增益达到预设阈值且可能改变下一步决策时，才向医生提示可考虑补充的具体检查及其可区分的竞争假设；不得以追求数据完整性为由盲目推荐所有检查。

典型流程为：先质控DTI；再调用PINN与扩散模拟模块，在明确边界条件下估计脑区 D^* 的参数区间；随后结合 $A\beta$ /Tau、影像及RAG证据，比对AQP4异常、基质重塑或血管因素等竞争性解释。若关键证据缺失，第六项工具给出带预期信息增益的检查建议；最后生成包含置信区间、引用定位、反证条件和后续实验的假设报告。预测、关联与因果效应仍须分别报告；智能体提供的是可计算的假设空间，而非因果结论。

5.4 临床交互与人机协同

临床嵌入的目标应是降低而非转移医生的认知负荷。系统宜在阅片、病历汇总或多学科讨论的既有节点提供一页式输出：以自然语言说明当前最受支持的联合表型、关键替代解释和不可回答部分；以可点击证据图谱连接原始影像区域、实验室结果、文献段落与工具日志；以置信度热力图显示哪些脑区或证据边缘受到域偏移和缺失数据影响。输出应避免冗长的模型推理文本，并把是否建议补充检查何种假设将被区分预期信息增益置于同一可审核界面。

医生保有否决、修改和延后执行建议的权利。否决必须可选择结构化原因，例如影像伪影、临床语境不符、检查禁忌或证据不足；系统将此作为审计记录，而不是自动覆盖。经伦理审批、

去标识和前瞻性评估后，可将专家对建议质量、证据完整性和不当推荐的反馈用于RLHF或偏好优化，使模型学习报告措辞、停止规则和转诊边界；医疗问答中引入临床专家反馈可改善回答的对齐性，但不能替代临床效用与安全性验证^[20]。不得将单次医生选择当作未经验证的生物学真值。高风险输出应采用双人复核或MDT确认，并保留人工修改前后的版本。

5.5 假设生成与Agent评估体系

常规分类准确率不足以评估以假设生成和验证编排为目标的医疗智能体。评估应在冻结的工具版本、独立外部队列和预先注册的任务集上进行，并至少覆盖以下指标：

1. **证据追溯准确率（无幻觉率）**：报告中可定位至原始数据、文献或工具日志的关键主张比例，以及引用是否真实支持该主张；应由领域专家盲法核验，不以文字流畅性替代证据正确性。
2. **假设可证伪性/可验证性评分**：衡量假设是否明确给出可观测终点、竞争解释、反证条件、样本/动物分层和可执行实验Protocol。不能转化为预注册实验或无法被潜在结果推翻的陈述，应判为低分。
3. **物理一致性**：检验PINN反演参数是否违反非负性、质量守恒、边界条件、已知生理范围或稳定性约束；同时报告方程残差、后验覆盖率和对边界条件的敏感性，而非只报告拟合误差。
4. **信息增益有效性**：比较推荐检查与常规路径在真实随访中减少假设不确定性、改变恰当临床决策或避免低价值检查的比例；须将增益与成本、风险、等待时间和患者负担共同评价。
5. **人机协同性与安全性**：记录医生接受、否决和修正建议的原因，测量完成任务时间、认知负荷、关键遗漏率 and 不当自动化依赖；对域外、缺失模态和证据冲突病例分别报告拒答或升级人工复核的表现。

这些指标将生成得像转化为可追溯、可反驳、受物理约束且在临床流程中有净收益的可审计要求。

6 挑战、伦理与展望

ECS研究的核心难题是跨尺度：微观结构、示踪动力学、宏观影像和临床结局不能简单等同。多中心协作需统一元数据、扫描协议、质控阈值和标注规范。对高风险输出，应设置域外检测、关键模态缺失拦截、人工复核队列和模型漂移监控。真实临床数据还应遵循最小必要、去标识、分级授权、加密传输和可审计访问原则。监管上，软件的预期用途、目标人群、输入边界、更新策略和临床评价计划必须清晰界定^[10]。

未来优先事项包括建立可复用的ECS数据标准；以动物/类器官与人群队列分层验证机制假设；开展跨中心外部验证；并将可解释性从事后可视化推进到可追溯证据、失败报告和可复现分析。走向临床的必要条件不是增加更多模态或更长的自动报告，而是让UQ明确系统何时不应外推，让人在回路的工作流明确谁能否决和如何升级，以及用假设生成专属评估体系证明推荐带来可测量的净临床收益。

7 结论

以AD的ECS/类淋巴清除障碍为例，多模态大模型可将宏观扩散代理、 $A\beta$ /Tau标志物、AQP4相关微观证据及病程信息组织为可检验的联合表型；跨尺度数字孪生智能体则可进一步编排质控、物理反演、空间组学、因果图和信息增益工具，提出具有明确反证条件的机制假设。其终极价值不在于以单一或跨尺度指标给出确定性诊断，而在于构建受物理规律与临床证据约束的假设生成与验证空间。通过量化域偏移不确定性、审慎处理缺失模态并保持医生的最终判断权，建设性假设才可能逐步转化为可信的科研与临床辅助能力；这一转化仍以独立外部验证、前瞻性研究和严格数据治理为前提。

A 可复现代码

本附录收录生成合成数据、训练PINN、训练ECS样式分割基线、执行消融实验和绘制结果的源文件。运行顺序为数据生成、PINN或分割演示、消融实验、结果绘制。运行环境需要Python、NumPy、PyTorch、OpenCV、Matplotlib和Pandas。代码中的字符串与注释按源文件原样保留。

A.1 运行配置

Listing 1: config.yaml

```
seed: 20260718
output_dir: results
simulation:
  nx: 80
  nt: 24
  noise_std: 0.015
  alpha: 0.20
  diffusion: 0.018
  velocity: 0.12
  clearance: 0.02
  regions: [caudate, thalamus]
pinn:
  epochs: 1200
  learning_rate: 0.002
  collocation_points: 600
segmentation:
  image_size: 64
  train_samples: 96
  epochs: 10
  learning_rate: 0.001
```

A.2 合成数据生成

Listing 2: dataset_generator.py

```
"""Generate synthetic TB-MRI tracer arrays and ECS microscopy-like slices.
```

```

No animal, human, or public image is read. Parameters are dimensionless
teaching settings derived from the ECS advection-diffusion formulation.
"""
from pathlib import Path
import numpy as np
import cv2

ROOT = Path(__file__).resolve().parent

def tracer_series(nx=80, nt=24, diffusion=0.018, velocity=0.12, alpha=0.20, noise_std=0.015):
    """Return [time, distance] synthetic concentration with region-dependent D."""
    x = np.linspace(0, 1, nx); t = np.linspace(0.02, 0.9, nt)
    xx, tt = np.meshgrid(x, t)
    d_map = diffusion * np.where(xx < 0.5, 1.0, 0.72) # two synthetic regions
    width = 0.03 + 2*d_map*tt
    c = alpha*np.exp(-(xx-0.25-velocity*tt)**2/(2*width))*np.sqrt(0.03/width)
    return x, t, np.clip(c + noise_std*np.random.randn(*c.shape), 0, None), d_map

def ecs_slice(size=64):
    """Create dark cell-like ellipses; remaining bright pixels are synthetic ECS."""
    image = np.full((size,size), .72, np.float32); cells = np.zeros_like(image)
    for _ in range(np.random.randint(5,10)):
        center = tuple(np.random.randint(6,size-6,2)); axes = tuple(np.random.randint(4,12,2))
        angle = float(np.random.randint(180))
        cv2.ellipse(image, center, axes, angle, 0, 360, .18, -1)
        cv2.ellipse(cells, center, axes, angle, 0, 360, 1, -1)
    return np.clip(image+.08*np.random.randn(size,size),0,1).astype(np.float32), 1-cells

def main():
    np.random.seed(20260718); out=ROOT/'synthetic_data'; out.mkdir(exist_ok=True)
    x,t,c,d=tracer_series(); np.savez(out/'tb_mri_synthetic.npz', x=x,t=t,concentration=c,
        diffusion_map=d)
    images,masks=zip(*(ecs_slice() for _ in range(120)))
    np.savez(out/'ecs_slices_synthetic.npz', images=np.stack(images), masks=np.stack(masks))
    print(f'Wrote synthetic datasets to {out}')

if __name__ == '__main__': main()

```

A.3 PINN示踪剂反演演示

Listing 3: pinn_ecs.py

```

"""Lightweight PINN demonstration for a 1-D ECS tracer inverse problem.

It trains only on synthetic concentration data created inside this file. The
estimated parameters are demonstration outputs, not measurements of brain ECS.
"""
from pathlib import Path

```

```

import numpy as np
import matplotlib.pyplot as plt
import torch
from torch import nn

torch.manual_seed(7)
np.random.seed(7)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
OUT = Path(__file__).resolve().parent / "results"
OUT.mkdir(exist_ok=True)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(2, 48), nn.Tanh(), nn.Linear(48, 48), nn.Tanh(), nn.
            Linear(48, 1))
    def forward(self, z): return self.net(z)

def analytic(x, t, d, v, alpha=1.0):
    """Gaussian tracer plume for  $C_t = D C_{xx} - v C_x$  on an open 1-D domain."""
    width = 0.03 + 2.0 * d * t
    return alpha * torch.exp(-((x - 0.25 - v*t)**2) / (2*width)) * torch.sqrt(0.03 / width)

def deriv(y, x): return torch.autograd.grad(y, x, torch.ones_like(y), create_graph=True)[0]

def main(epochs=1200):
    true_d, true_v, true_alpha = 0.018, 0.12, 0.20 # dimensionless synthetic ground truth
    model = MLP().to(DEVICE)
    log_d = nn.Parameter(torch.tensor([-3.5], device=DEVICE))
    velocity = nn.Parameter(torch.tensor([0.02], device=DEVICE))
    log_alpha = nn.Parameter(torch.tensor([-1.4], device=DEVICE))
    opt = torch.optim.Adam(list(model.parameters()) + [log_d, velocity, log_alpha], lr=2e-3)
    x_data = torch.rand(180, 1, device=DEVICE) * 0.9 + 0.05
    t_data = torch.rand(180, 1, device=DEVICE) * 0.8 + 0.05
    c_data = analytic(x_data, t_data, true_d, true_v, true_alpha).detach() + 0.01*torch.randn_like(
        x_data)
    for epoch in range(epochs):
        x_f = torch.rand(600, 1, device=DEVICE, requires_grad=True)
        t_f = torch.rand(600, 1, device=DEVICE, requires_grad=True)
        pred = model(torch.cat([x_f, t_f], 1))
        c_t = deriv(pred, t_f)
        c_x = deriv(pred, x_f)
        c_xx = deriv(c_x, x_f)
        d = torch.exp(log_d)
        physics = c_t - d*c_xx + velocity*c_x
        data_loss = ((model(torch.cat([x_data, t_data], 1)) - c_data)**2).mean()
        initial = model(torch.cat([x_f, torch.zeros_like(t_f)], 1))
        initial_loss = ((initial - analytic(x_f, torch.zeros_like(t_f), true_d, true_v, true_alpha
            ))**2).mean()

```

```

        loss = data_loss + physics.square().mean() + initial_loss
        opt.zero_grad(); loss.backward(); opt.step()
        if epoch % 300 == 0: print(f"epoch={epoch:4d} loss={loss.item():.4e} D={d.item():.4f} v={
            velocity.item():.4f} alpha={torch.exp(log_alpha).item():.3f}")
x = torch.linspace(0, 1, 200, device=DEVICE)[: , None]
fig, ax = plt.subplots(figsize=(6, 3.2))
for time in (0.2, 0.5, 0.8):
    t = torch.full_like(x, time)
    with torch.no_grad(): y = model(torch.cat([x, t], 1)).cpu().numpy()
    ax.plot(x.cpu(), y, label=f"PINN t={time}")
ax.set(xlabel="normalized distance", ylabel="tracer concentration", title="Synthetic ECS PINN
    result")
ax.legend(fontsize=8); fig.tight_layout(); fig.savefig(OUT / "pinn_tracer_profiles.png", dpi
    =160)
print(f"Synthetic estimate: D={torch.exp(log_d).item():.4f}, v={velocity.item():.4f}, alpha={
    torch.exp(log_alpha).item():.3f}")

if __name__ == "__main__": main()

```

A.4 ECS样式分割基线

Listing 4: ecs_net.py

```

"""Shape-aware dual-branch ECS segmentation demo using generated toy images.

This is an educational ECS-Net-style baseline: it does not claim to reproduce a
published architecture or use a biological EM dataset.
"""
from pathlib import Path
import numpy as np
import cv2
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset

torch.manual_seed(9); np.random.seed(9)
OUT = Path(__file__).resolve().parent / "results"; OUT.mkdir(exist_ok=True)

def sample(n=64):
    image = np.zeros((n,n), np.float32); mask = np.zeros((n,n), np.float32)
    # Bright narrow corridors mimic ECS; dark ellipses mimic cellular profiles.
    image[:] = .72
    for _ in range(np.random.randint(5, 10)):
        c = tuple(np.random.randint(5, n-5, 2)); axes = tuple(np.random.randint(4, 12, 2))
        cv2.ellipse(image, c, axes, float(np.random.randint(180)), 0, 360, .18, -1)
        cv2.ellipse(mask, c, axes, float(np.random.randint(180)), 0, 360, 1, -1)
    ecs = 1-mask
    image += .08*np.random.randn(n,n).astype(np.float32)

```

```

    return np.clip(image,0,1), ecs

class ToyECS(Dataset):
    def __init__(self, count): self.items=[sample() for _ in range(count)]
    def __len__(self): return len(self.items)
    def __getitem__(self, i):
        a,b=self.items[i]; return torch.tensor(a)[None], torch.tensor(b)[None]

class ECSDualNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder=nn.Sequential(nn.Conv2d(1,16,3,padding=1),nn.ReLU(),nn.Conv2d(16,16,3,padding
            =1),nn.ReLU())
        self.mask=nn.Conv2d(16,1,1); self.edge=nn.Conv2d(16,1,1)
    def forward(self,x):
        f=self.encoder(x); return self.mask(f),self.edge(f)

def dice(logits, y):
    p=torch.sigmoid(logits); return 1-(2*(p*y).sum()+1)/((p+y).sum()+1)

def contrastive_feature_loss(features, mask):
    """Lightweight contrastive surrogate: separate mean ECS/cell feature vectors."""
    ecs = (features * mask).sum((0,2,3)) / (mask.sum((0,2,3))+1e-6)
    cell_mask = 1-mask
    cell = (features * cell_mask).sum((0,2,3)) / (cell_mask.sum((0,2,3))+1e-6)
    return torch.exp(-torch.norm(ecs-cell))

def main(epochs=10):
    device="cuda" if torch.cuda.is_available() else "cpu"; model=ECSDualNet().to(device)
    loader=DataLoader(ToyECS(96),batch_size=12,shuffle=True); opt=torch.optim.Adam(model.
        parameters(),1e-3)
    bce=nn.BCEWithLogitsLoss()
    for e in range(epochs):
        for x,y in loader:
            x,y=x.to(device),y.to(device)
            edge=(torch.abs(y[:, :, 1:] - y[:, :, :-1])>0).float(); edge=torch.nn.functional.pad(edge
                , (0,0,0,1))
            seg,shape=model(x); features=model.encoder(x)
            loss=bce(seg,y)+dice(seg,y)+0.3*bce(shape,edge)+0.05*contrastive_feature_loss(features,
                y)
            opt.zero_grad();loss.backward();opt.step()
            print(f"epoch={e+1:02d} loss={loss.item():.4f}")
    x,y=ToyECS(1)[0];
    with torch.no_grad(): p=torch.sigmoid(model(x[None].to(device))[0])[0,0].cpu().numpy()
    fig,ax=plt.subplots(1,3,figsize=(8,2.6))
    for a,z,title in zip(ax,[x[0],y[0],p],["synthetic slice","ECS label","prediction"]): a.imshow(
        z,cmap="gray");a.set_title(title);a.axis("off")
    fig.tight_layout();fig.savefig(OUT/"ecs_net_segmentation.png",dpi=160)

```



```
if __name__ == "__main__": main()
```

A.5 消融实验

Listing 5: ablation_experiment.py

```
"""Run a synthetic-data ablation for the ECS PINN demonstration.

This experiment uses only arrays generated by dataset_generator.py. It reports
optimization behavior on a known synthetic system; it is not a biological or
clinical performance study.
"""
from pathlib import Path
import csv
import time
import numpy as np
import torch
from torch import nn

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results"
OUT.mkdir(exist_ok=True)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

class Network(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(2, 32), nn.Tanh(), nn.Linear(32, 32), nn.Tanh(), nn.Linear(32, 1)
        )
    def forward(self, z):
        return self.layers(z)

def analytic(x, t, d, v, alpha):
    width = 0.03 + 2.0 * d * t
    return alpha * torch.exp(-(x - 0.25 - v * t) ** 2 / (2 * width)) * torch.sqrt(0.03 / width)

def grad(y, x):
    return torch.autograd.grad(y, x, torch.ones_like(y), create_graph=True)[0]

def run_one(name, use_pde, use_initial, use_region_prior, seed, epochs=100):
    """Train one configuration and return held-out synthetic-data metrics."""
    torch.manual_seed(seed); np.random.seed(seed)
    true_d, true_v, true_alpha = 0.018, 0.12, 0.20
    model = Network().to(DEVICE)
    log_d = nn.Parameter(torch.tensor([-3.7], device=DEVICE))
    velocity = nn.Parameter(torch.tensor([0.04], device=DEVICE))
    log_alpha = nn.Parameter(torch.tensor([-1.5], device=DEVICE))
    optimizer = torch.optim.Adam([*model.parameters(), log_d, velocity, log_alpha], lr=2e-3)
```

```

# Train and held-out samples are both generated from the known synthetic equation.
x_train = torch.rand(120, 1, device=DEVICE) * 0.9 + 0.05
t_train = torch.rand(120, 1, device=DEVICE) * 0.8 + 0.05
y_train = analytic(x_train, t_train, true_d, true_v, true_alpha).detach() + 0.01 * torch.
    randn_like(x_train)
x_test = torch.rand(180, 1, device=DEVICE) * 0.9 + 0.05
t_test = torch.rand(180, 1, device=DEVICE) * 0.8 + 0.05
y_test = analytic(x_test, t_test, true_d, true_v, true_alpha).detach()
started = time.perf_counter()
for _ in range(epochs):
    d, alpha = torch.exp(log_d), torch.exp(log_alpha)
    pred = alpha * model(torch.cat([x_train, t_train], 1))
    loss = ((pred - y_train) ** 2).mean()
    if use_initial:
        x0 = torch.rand(80, 1, device=DEVICE)
        initial = alpha * model(torch.cat([x0, torch.zeros_like(x0)], 1))
        loss = loss + ((initial - analytic(x0, torch.zeros_like(x0), true_d, true_v, true_alpha)
            )) ** 2).mean()
    x_f = torch.rand(100, 1, device=DEVICE, requires_grad=True)
    t_f = torch.rand(100, 1, device=DEVICE, requires_grad=True)
    c = alpha * model(torch.cat([x_f, t_f], 1))
    if use_pde:
        residual = grad(c, t_f) - d * grad(grad(c, x_f), x_f) + velocity * grad(c, x_f)
        loss = loss + residual.square().mean()
    if use_region_prior:
        # Synthetic prior: regularize D toward the known training-domain scale, not a real
        # measurement.
        loss = loss + 0.03 * (d - true_d).square()
    optimizer.zero_grad(); loss.backward(); optimizer.step()
elapsed = time.perf_counter() - started
with torch.no_grad():
    test_mse = ((torch.exp(log_alpha) * model(torch.cat([x_test, t_test], 1)) - y_test) ** 2).
        mean().item()
# Calculate residual after training for comparable quality control.
x_f = torch.rand(120, 1, device=DEVICE, requires_grad=True); t_f = torch.rand(120, 1, device=
    DEVICE, requires_grad=True)
c = torch.exp(log_alpha) * model(torch.cat([x_f, t_f], 1))
residual = grad(c, t_f) - torch.exp(log_d) * grad(grad(c, x_f), x_f) + velocity * grad(c, x_f)
return dict(configuration=name, seed=seed, test_mse=test_mse,
    d_relative_error=abs(torch.exp(log_d).item()-true_d)/true_d,
    v_relative_error=abs(velocity.item()-true_v)/true_v,
    alpha_absolute_error=abs(torch.exp(log_alpha).item()-true_alpha),
    pde_residual=float(residual.square().mean().sqrt().detach().cpu()), seconds=elapsed
)

def main():
    configurations = [
        ("data_only", False, False, False), ("data_pde", True, False, False),
        ("data_pde_initial", True, True, False), ("data_pde_initial_prior", True, True, True),

```

```

        ("full_qc", True, True, True),
    ]
    rows = []
    for name, pde, initial, prior in configurations:
        for seed in (17, 29, 43):
            row = run_one(name, pde, initial, prior, seed)
            rows.append(row); print(name, seed, f"mse={row['test_mse']:.5f}", f"Derr={row['d_relative_error']:.3f}")
    path = OUT / "ablation_results.csv"
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=rows[0].keys()); writer.writeheader(); writer.writerows(rows)
    print(f"Wrote {path}")

if __name__ == "__main__": main()

```

A.6 结果绘制

Listing 6: visualization.py

```

"""Visualize only synthetic outputs created by this project."""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import csv

ROOT=Path(__file__).resolve().parent
def main():
    out=ROOT/'results'; out.mkdir(exist_ok=True)
    data=np.load(ROOT/'synthetic_data'/ 'tb_mri_synthetic.npz'); x,t,c,d=data['x'],data['t'],data['concentration'],data['diffusion_map']
    fig,ax=plt.subplots(1,2,figsize=(8,3))
    ax[0].imshow(d,aspect='auto',cmap='viridis'); ax[0].set(title='Synthetic D map',xlabel='distance',ylabel='time')
    for i in (0,len(t)//2,-1): ax[1].plot(x,c[i],label=f't={t[i]:.2f}')
    ax[1].set(title='Synthetic TB-MRI tracer',xlabel='normalized distance',ylabel='concentration')
    ; ax[1].legend(fontsize=8)
    fig.tight_layout(); fig.savefig(out/'ecs_parameter_heatmap_and_series.png',dpi=160)
    result_path = out/'ablation_results.csv'
    if result_path.exists():
        with result_path.open(encoding='utf-8') as handle: rows=list(csv.DictReader(handle))
        names=list(dict.fromkeys(row['configuration'] for row in rows))
        means=[np.mean([float(row['d_relative_error']) for row in rows if row['configuration']==name]) for name in names]
        stds=[np.std([float(row['d_relative_error']) for row in rows if row['configuration']==name]) for name in names]
        fig,ax=plt.subplots(figsize=(7,3.4)); ax.bar(names,means,yerr=stds,capsize=3,color='#5B8FF9')

```

```

    ax.set(ylabel='relative error of synthetic D',title='Synthetic PINN ablation (mean +/- SD,
            n=3)'); ax.tick_params(axis='x',rotation=22)
    fig.tight_layout(); fig.savefig(out/'ablation_results.png',dpi=160)
else:
    print('No ablation_results.csv found; run ablation_experiment.py first.')
if __name__=='__main__': main()

```

参考文献

- [1] SYKOVA E, NICHOLSON C. Diffusion in brain extracellular space[J/OL]. *Physiological Reviews*, 2008, 88(4):1277-1340. DOI: [10.1152/physrev.00027.2007](https://doi.org/10.1152/physrev.00027.2007).
- [2] NICHOLSON C, SYKOVA E. Extracellular space structure revealed by diffusion analysis [J/OL]. *Trends in Neurosciences*, 1998, 21(5):207-215. DOI: [10.1016/S0166-2236\(98\)01261-2](https://doi.org/10.1016/S0166-2236(98)01261-2).
- [3] TONNESEN J, INAVALLI V, NAGERL U V. Super-resolution imaging of the extracellular space in living brain tissue[J/OL]. *Cell*, 2018, 172(5):1108-1121.e15. DOI: [10.1016/j.cell.2018.02.007](https://doi.org/10.1016/j.cell.2018.02.007).
- [4] ILIFF J J, WANG M, LIAO Y, et al. A paravascular pathway facilitates CSF flow through the brain parenchyma and the clearance of interstitial solutes, including amyloid beta[J/OL]. *Science Translational Medicine*, 2012, 4(147):147ra111. DOI: [10.1126/scitranslmed.3003748](https://doi.org/10.1126/scitranslmed.3003748).
- [5] MESTRE H, MORI Y, NEDERGAARD M. The brain's glymphatic system: current controversies[J/OL]. *Trends in Neurosciences*, 2020, 43(7):458-466. DOI: [10.1016/j.tins.2020.04.003](https://doi.org/10.1016/j.tins.2020.04.003).
- [6] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems: vol. 33. 2020: 9459-9474.
- [7] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing reasoning and acting in language models [C]//International Conference on Learning Representations. 2023.
- [8] MOOR M, BANERJEE O, ABAD Z S, et al. Foundation models for generalist medical artificial intelligence[J/OL]. *Nature*, 2023, 616:259-265. DOI: [10.1038/s41586-023-05881-4](https://doi.org/10.1038/s41586-023-05881-4).
- [9] TOPOL E J. High-performance medicine: the convergence of human and artificial intelligence[J/OL]. *Nature Medicine*, 2019, 25:44-56. DOI: [10.1038/s41591-018-0300-7](https://doi.org/10.1038/s41591-018-0300-7).
- [10] U.S. Food and Drug Administration. Artificial Intelligence/Machine Learning-Based Software as a Medical Device Action Plan[EB/OL]. 2021 [2026-07-16]. <https://www.fda.gov/media/145022/download>.
- [11] ZEPPENFELD D M, SIMON M, HASWELL J D, et al. Association of perivascular localization of aquaporin-4 with cognition and Alzheimer disease in aging brains[J/OL]. *JAMA Neurology*, 2017, 74(1):91-99. DOI: [10.1001/jamaneurol.2016.4370](https://doi.org/10.1001/jamaneurol.2016.4370).

- [12] XIE L, KANG H, XU Q, et al. Sleep drives metabolite clearance from the adult brain[J/OL]. *Science*, 2013, 342(6156): 373-377. DOI: [10.1126/science.1241224](https://doi.org/10.1126/science.1241224).
- [13] TAOKA T, NAGANAWA S. Glymphatic imaging using diffusion tensor image analysis along the perivascular space (DTI-ALPS) in Alzheimer's disease[J/OL]. *Japanese Journal of Radiology*, 2021, 39: 110-118. DOI: [10.1007/s11604-020-00997-1](https://doi.org/10.1007/s11604-020-00997-1).
- [14] KIPF T N, WELING M. Semi-supervised classification with graph convolutional networks [C]//International Conference on Learning Representations. 2017.
- [15] SHAFER G. *A Mathematical Theory of Evidence*[M]. Princeton, NJ: Princeton University Press, 1976.
- [16] GAWLIKOWSKI J, TASSI C, ALI M, et al. A survey of uncertainty in deep neural networks [J/OL]. *Artificial Intelligence Review*, 2023, 56: 1513-1589. DOI: [10.1007/s10462-023-10562-9](https://doi.org/10.1007/s10462-023-10562-9).
- [17] RAISSI M, PERDIKARIS P, KARNIADAKIS G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations[J/OL]. *Journal of Computational Physics*, 2019, 378: 686-707. DOI: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- [18] SATIJA R, FARRELL J A, GENNERT D, et al. Spatial reconstruction of single-cell gene expression data[J/OL]. *Nature Biotechnology*, 2015, 33: 495-502. DOI: [10.1038/nbt.3192](https://doi.org/10.1038/nbt.3192).
- [19] PALLA G, SPITZER H, KLEIN M, et al. Squidpy: a scalable framework for spatial omics analysis[J/OL]. *Nature Methods*, 2022, 19: 171-178. DOI: [10.1038/s41592-021-01358-2](https://doi.org/10.1038/s41592-021-01358-2).
- [20] SINGHAL K, AZIZI S, TU T, et al. Large language models encode clinical knowledge[J/OL]. *Nature*, 2023, 620: 172-180. DOI: [10.1038/s41586-023-06291-2](https://doi.org/10.1038/s41586-023-06291-2).